

Optimización de *Large Language Models* para la realización de tareas

Defensa del Trabajo de Fin de Grado

Daniel Serrano Alzamora

Grado en Matemáticas
Universidad de Alicante

Julio 2025

Índice

- 1 Introducción
- 2 Redes Neuronales
- 3 LLMs
- 4 Optimización en LLMs
- 5 Técnica LoRA
- 6 Caso práctico
 - Resultados y comparativas
- 7 Conclusiones
- 8 Trabajos futuros

Introducción

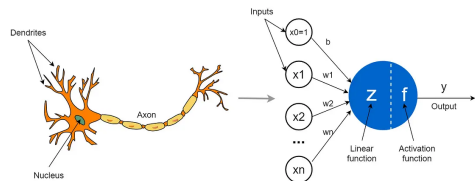
- Los *Large Language Models* (LLMs) son una herramienta central en IA moderna: destacan por su capacidad para comprender y generar texto.
- Tienen impacto en múltiples sectores: asistentes virtuales, programación, análisis de datos, etc.
- Sin embargo, presentan desafíos:
 - Alto coste computacional: difícil ejecutarlos en hardware limitado.
 - Reentrenamiento completo para cada tarea resulta inviable.
- Surgen técnicas como:
 - Cuantización y poda (*pruning*) para reducir recursos.
 - Adaptaciones eficientes como **LoRA**.
- **LoRA**: inserta matrices de bajo rango en modelos preentrenados, permitiendo adaptarlos a tareas concretas sin modificar todos sus parámetros.

Objetivos del trabajo

- Estudiar fundamentos matemáticos y computacionales de LLMs y técnicas de adaptación como LoRA.
- Analizar los desafíos de escalabilidad y adaptación.
- Comparar LLMs adaptados con LoRA frente a algoritmos clásicos de *ML* en tareas de predicción o clasificación.

Preliminares: Redes Neuronales

- Las Redes Neuronales Artificiales (ANNs) están inspiradas en el cerebro humano.
- Son eficaces en tareas como:
 - Clasificación y regresión.
 - Procesamiento de imágenes.
 - Análisis de lenguaje natural.
- Aproximan funciones no lineales: clave en el aprendizaje automático moderno.
- Son la base de arquitecturas complejas como los **LLMs**.



Comparación entre una neurona biológica y una artificial.

Modelo de Neurona Artificial (Perceptrón)

Entrada: $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, **Pesos:** $\mathbf{w} = (w_1, \dots, w_n) \in \mathbb{R}^n$, **Sesgo:** $b \in \mathbb{R}$

Potencial de activación:

$$z = \mathbf{w} \cdot \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b$$

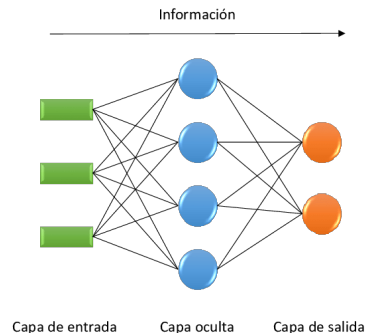
Salida:

$$a = \phi(z)$$

Funciones de activación comunes:

- Sigmoides: $\phi(z) = \frac{1}{1 + e^{-z}}$
- Tangente hiperbólica: $\phi(z) = \tanh(z)$
- ReLU: $\phi(z) = \max(0, z)$

Arquitecturas de Redes Neuronales



Ejemplo esquemático de una red neuronal

- Una red neuronal se compone de múltiples capas de neuronas conectadas. Tipos principales:
 - **Redes neuronales feedforward (FNN):** flujo unidireccional, sin ciclos.
 - **Redes neuronales profundas (DNN):** FNN con múltiples capas ocultas.
 - **Redes neuronales recurrentes (RNN):** con ciclos; adecuadas para secuencias.
 - **Redes convolucionales (CNN):** extraen patrones locales, útiles en imágenes y texto.
- Modelos modernos como los **Transformers** combinan atención y paralelismo para superar limitaciones de las RNN.

Función de Coste y Optimización

- Se minimiza una **función de pérdida** $\mathcal{L}(y, \hat{y})$. Ejemplos comunes:

- Clasificación: entropía cruzada

$$\mathcal{L}(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

- Regresión: error cuadrático medio

$$\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2$$

- Objetivo: minimizar la pérdida media

$$Loss = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(y^{(i)}, \hat{y}^{(i)})$$

- Se usa **descenso del gradiente**:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} Loss$$

$$b \leftarrow b - \eta \nabla_b Loss$$

Con tasa de aprendizaje $\eta > 0$.

Retropropagación y Entrenamiento

- Se usa **backpropagation** para computar gradientes eficientemente.
- Aplica la **regla de la cadena** a través de las capas.

Gradientes:

$$\frac{\partial \mathcal{L}}{\partial w_{ji}} = \delta_j \cdot a_i, \quad \frac{\partial \mathcal{L}}{\partial b_j} = \delta_j$$

Errores:

- Capa de salida: $\delta_j = \frac{\partial \mathcal{L}}{\partial a_j} \cdot \phi'(z_j)$
- Capas ocultas: $\delta_j = (\sum_k w_{kj} \delta_k) \cdot \phi'(z_j)$

Observación:

- Las redes neuronales son potentes, pero requieren muchos datos, buen ajuste de hiperparámetros y presentan baja interpretabilidad.

Large Language Models (LLMs)

- Los LLMs son modelos de aprendizaje profundo entrenados sobre grandes corpus textuales para procesar y generar lenguaje natural.
- Basados en arquitecturas altamente parametrizadas.
- Definición formal:

$$f_{\theta} : \mathcal{X}^n \rightarrow \mathcal{Y}, \quad \theta \in \mathbb{R}^d$$

- Modelan la distribución de probabilidad de secuencias:

$$P(x_t \mid x_1, x_2, \dots, x_{t-1}; \theta)$$

- Permiten tareas como:
 - Completado de texto
 - Traducción
 - Generación de lenguaje
 - Clasificación o análisis semántico

Tokenización y Representaciones Vectoriales

- Los LLMs no procesan texto directamente, sino vectores numéricos.
- **Tokenización:** divide el texto en unidades llamadas *tokens*.
- Tipos de tokens: palabra, subpalabra, carácter o símbolos especiales.
- Cada *token* se transforma en un vector mediante una capa de **embeddings**.
- Los **embeddings** capturan propiedades sintácticas y semánticas del lenguaje.
- La elección de la tokenización afecta al rendimiento y capacidad de generalización del modelo.

Algoritmos de Tokenización

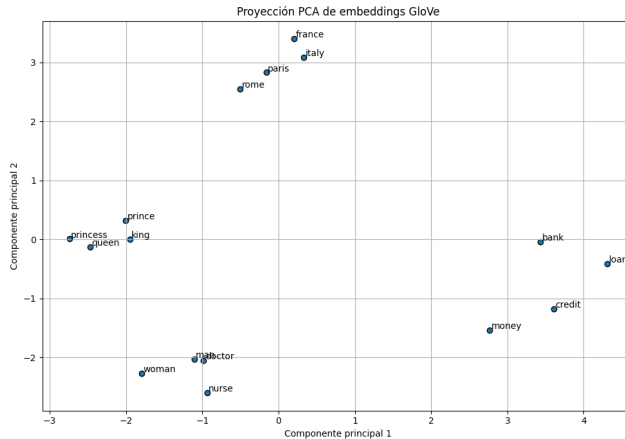
- Los LLMs modernos usan tokenizadores por **subpalabras**, lo que mejora eficiencia y generalización.
- **Byte Pair Encoding (BPE)**. Usado en GPT-2, GPT-3, LLaMA:
 - Fusiona pares de caracteres más frecuentes y es eficiente con palabras raras.
 - [‘‘Transform’’, ‘‘ers’’, ‘‘ are’’, ‘‘ powerful’’]
- **WordPiece**. Usado en BERT:
 - Fusiona pares de subpalabras maximizando su verosimilitud de lenguaje pero requiere preentrenar un modelo de lenguaje.
 - [‘‘Transform’’, ‘‘##ers’’, ‘‘are’’, ‘‘powerful’’]
- **Unigram Language Model**. Usado en T5, mT5, Flan-T5 (modelo de este trabajo):
 - Selecciona la mejor secuencia de tokens de forma probabilística y es robusto ante ruido; óptimo para tareas multilingües y generativas.
 - [‘‘Trans’’, ‘‘former’’, ‘‘s’’, ‘‘are’’, ‘‘powerful’’] (*posible*)

Procesamiento Post-tokenización y Embeddings

- Tras la tokenización, cada *token* t_i se convierte en un ID entero usando un vocabulario fijo ($|V| \approx 30k - 50k$).
- Secuencias de longitud variable se normalizan con:
 - **Padding**: se añade [PAD] hasta una longitud fija.
 - **Truncamiento**: se recortan secuencias largas.
- **Capa de embeddings**: transforma cada ID t_i en un vector denso $x_i \in \mathbb{R}^d$:

$$x_i = E(t_i), \quad E \in \mathbb{R}^{|V| \times d}$$

- Los embeddings se entrenan junto al modelo y capturan similitud semántica entre tokens.



Proyección PCA de embeddings preentrenados (GloVe). Palabras relacionadas semánticamente tienden a agruparse.

Codificación Posicional

- Los *Transformers* no tienen conciencia del orden; se añade un vector posicional p_i a cada embedding:

$$z_i = x_i + p_i$$

- Principales enfoques:

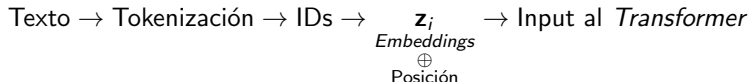
- Codificación sinusoidal:**

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right)$$

Generaliza bien a secuencias más largas que las vistas en entrenamiento.

- Codificación aprendible:** los vectores p_i son parámetros entrenables. Más flexible en tareas concretas.

Flujo completo de entrada:



Preentrenamiento de LLMs

- Preentrenamiento auto-supervisado sobre grandes corpus textuales.
- Permite aprender representaciones lingüísticas generales.
- Arquitecturas principales:
 - **Encoder-only**: atención bidireccional, útil en comprensión (ej. BERT).
 - **Decoder-only**: atención causal, orientado a generación (ej. GPT).
 - **Encoder-decoder**: entrada-salida, ideal para traducción o resumen (ej. T5).

Comparación de paradigmas de preentrenamiento en LLMs

Paradigma	Arquitectura	Contexto de atención	Objetivo preentrenamiento	Aplicaciones típicas	Ejemplos
Modelado de lenguaje causal (CLM)	<i>Decoder-only</i>	Unidireccional (solo pasado)	Predecir el siguiente <i>token</i>	Generación de texto	GPT, LLaMA
Modelado de lenguaje enmascarado (MLM)	<i>Encoder-only</i>	Bidireccional completo	Predecir <i>tokens</i> ocultos	Clasificación, codificación, comprensión	BERT, RoBERTa
<i>Denoising</i> / <i>Span masking</i>	<i>Encoder-Decoder</i>	Enmascaramiento de <i>spans</i> y reconstrucción secuencial	Reconstruir texto corrupto (<i>span corruption</i>)	Tareas mixtas (generación + entrada-salida supervisada)	T5, Flan-T5

Arquitectura de los LLMs

- Los LLMs modernos se basan en el **Transformer**, que reemplaza RNNs por mecanismos de atención paralelizables.
- Permite modelar dependencias a largo plazo con mayor eficiencia.

LLMs usan principalmente la pila de **decoders**, compuesta por capas que incluyen:

1 Self-Attention:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

2 Multi-Head Attention (MHA):

$$\text{MHA} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Arquitectura de los LLMs

3 Feed-Forward Network (FFN):

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

- Se aplica a cada token de forma independiente.
- Suele tener una dimensión intermedia $d_{\text{ff}} \gg d$.

4 Residual Connections + LayerNorm:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

- Mejora la propagación del gradiente y evita dependencias entre ejemplos de un mismo *batch*.

Nota: *LayerNorm* normaliza por características ($\mathbb{R}^d$), no por batch, lo cual es clave para secuencias con padding y estabilidad en generación.

Resumen: Esta arquitectura favorece una optimización estable y una generalización eficaz en tareas de modelado del lenguaje.

Salida del Modelo

- Tras las capas del decoder, se obtiene un vector contextualizado $z_t \in \mathbb{R}^d$ para el *token* en la posición t .
- Se proyecta al espacio del vocabulario y se aplica *softmax*:

$$P(x_t \mid x_{<t}) = \text{softmax}(Wz_t + b)$$

$$\text{softmax}(u_i) = \frac{\exp(u_i)}{\sum_{j=1}^{|V|} \exp(u_j)}$$

- Se genera un *token* por iteración (autoregresivo) hasta una longitud máxima predefinida o un token especial de parada <eos>.
- La salida se concatena a la entrada y se reintroduce en el modelo.

Parámetros de Ajuste de la Salida

Técnicas de muestreo:

- **Temperature scaling:**

- Ajusta la aleatoriedad de la salida: menor temperatura produce texto más determinista.
- Temperaturas altas generan mayor diversidad pero pueden reducir la coherencia.

- **Top- k sampling:**

- Se limita a los k *tokens* más probables.
- Renormaliza la distribución dentro del conjunto $\mathcal{K}$.

- **Top- p sampling (nucleus):**

- Se seleccionan los *tokens* cuya probabilidad acumulada $\leq p$.
- Tamaño de $\mathcal{P}$ dinámico según la forma de la distribución.

Nota: En la práctica se combinan técnicas (ej. *top-k* + temperatura) para ajustar coherencia y diversidad.

Optimización en LLMs: Retos y Soluciones

- **Alto coste computacional y de memoria:**
 - Cuantización (8-bit vs 32-bit)
 - *Sparsity*: eliminación de conexiones irrelevantes
- **Riesgo de sobreajuste:**
 - Técnicas de regularización: Dropout, ℓ_2 , *data augmentation*
- **Adaptación eficiente a nuevas tareas:**
 - Fine-tuning eficiente (PEFT): ajusta solo una fracción de los parámetros
- **Obsolescencia del conocimiento:**
 - *Continual / Online Learning* con estrategias anti-*forgetting*
- **Reducción de latencia en inferencia:**
 - *Knowledge Distillation* y *Model Pruning*

Fine-Tuning de Modelos

- Tras el preentrenamiento, los LLMs se ajustan a tareas específicas mediante **fine-tuning**.
- **Full fine-tuning**: ajusta todos los parámetros del modelo base.
 - Alta capacidad de adaptación, pero muy costoso.
- **Fine-tuning supervisado**: se entrena con datos etiquetados para una tarea concreta.
- Problemas de escalabilidad motivan el desarrollo de métodos más eficientes: **Parameter-Efficient Fine-Tuning (PEFT)**.

Técnicas PEFT

Objetivo: adaptar un modelo preentrenado modificando solo una pequeña fracción de sus parámetros.

- **Adapters:**

- Capas entrenables insertadas entre las capas del modelo.
- Modelo base compartido entre tareas.

- **BitFit:**

- Ajusta solo los sesgos (*bias*). Deja los pesos principales congelados.
- Muestra resultados competitivos a pesar de su simplicidad.

- **Prefix Tuning:**

- Añade vectores entrenables al inicio de la entrada de cada capa que sirven de guía contextual que orienta la atención.

- **LoRA:**

- Inserta matrices de bajo rango entrenables en proyecciones lineales.
- Alta eficiencia, especialmente en modelos muy grandes.

Comparativa PEFT y LoRA

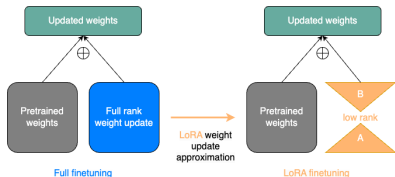
- Las técnicas PEFT permiten adaptar modelos con bajo coste computacional.
- **LoRA** destaca por:
 - Buena escalabilidad a modelos de gran tamaño.
 - Alto rendimiento con muy pocos parámetros entrenables.
- Estudios como Lialin et al. (2023) muestran sus ventajas frente a otras técnicas.

Method	%Trainable parameters	%Changed parameters	Evaluated on		
			<1B	<20B	>20B
Adapters	0.1 - 6	0.1 - 6	✓	✓	✓
AdaMix	0.1 - 0.2	0.1 - 0.2	✓	✗	✗
SparseAdapter	2.0	2.0	✓	✗	✗
BitFit	0.05 - 0.1	0.05 - 0.1	✓	✓	✓
DiffPruning	200	0.5	✓	✗	✗
Fish-Mask	0.01 - 0.5	0.01 - 0.5	✓	✓	✗
Prompt Tuning	0.1	0.1	✓	✓	✓
Prefix-Tuning	0.1 - 4.0	0.1 - 4.0	✓	✓	✓
IPT	56.0	56.0	✓	✗	✗
MAM Adapter	0.5	0.5	✓	✗	✗
Parallel Adapter	0.5	0.5	✓	✗	✗
Intrinsic SAID	0.001 - 0.1	100	✓	✓	✗
LoRa	0.01 - 0.5	~ 30	✓	✓	✓
DoRA	0.01 - 0.5	~ 30	✗	✓	✗
UniPELT	1.0	1.0	✓	✗	✗
Compacter	0.05-0.07	~ 0.1	✓	✓	✗
PHM Adapter	0.2	~ 1.0	✓	✗	✗
KronA	0.07	~ 30.0	✓	✗	✗
KronA ^B _{res}	0.07	~ 1.0	✓	✗	✗
(IA) ³	0.02	0.02	✗	✓	✗
Ladder Side-Tuning	7.5	7.5	✓	✓	✗
FAR	6.6-26.4	6.6-26.4	✓	✗	✗
S4-model	0.5	> 0.5	✓	✓	✗

Comparativa de parámetros entrenables en distintos métodos PEFT.

Low-Rank Adaptation (LoRA)

- **Problema:** el *fine-tuning* completo de LLMs es costoso e ineficiente.
- **Solución:** LoRA añade matrices de bajo rango entrenables al modelo, sin modificar los parámetros originales.



Ventajas de LoRA:

- Ahorro computacional
- Modularidad por tarea
- Preserva el modelo base

Comparativa: *full fine-tuning* vs LoRA

Formulación Matemática de LoRA

- Sea $W_0 \in \mathbb{R}^{d \times k}$ una matriz preentrenada.
- LoRA la modifica como:

$$W = W_0 + \frac{\alpha}{r}BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}$$

- Se evita actualizar W_0 , y solo se entrena la perturbación $\Delta W = BA$.
- Esto reduce la cantidad de parámetros entrenables de dk a $r(d + k)$.

Justificación Algebraica y Geométrica

- Cualquier matriz de rango r puede descomponerse como:

$$\Delta W = \sum_{i=1}^r \mathbf{b}_i \mathbf{a}_i^\top = BA$$

- Esto representa una actualización en un subespacio lineal de dimensión reducida.
- **Interpretación geométrica:**
 - Se restringe la modificación de W_0 a una subvariedad de dimensión $r(d + k)$.
 - Actúa como regularización implícita: menor riesgo de sobreajuste.
- Al mantener W_0 fijo se conserva el conocimiento previo, evitando *catastrophic forgetting*.

Ejemplo simple de LoRA: configuración inicial

Entrada:

$$x = \begin{bmatrix} 1,0 \\ 2,0 \\ -1,0 \end{bmatrix}$$

Matriz preentrenada (congelada):

$$W = \begin{bmatrix} 0,2 & -0,1 & 0,4 \\ 0,0 & 0,3 & -0,2 \\ -0,5 & 0,1 & 0,6 \end{bmatrix}$$

Adaptación LoRA ($r = 1$):

$$A = \begin{bmatrix} 0,1 & 0,2 & -0,3 \end{bmatrix} \quad B = \begin{bmatrix} 0,5 \\ -0,4 \\ 1,0 \end{bmatrix}$$

Corrección:

$$\Delta W = BA = \begin{bmatrix} 0,05 & 0,10 & -0,15 \\ -0,04 & -0,08 & 0,12 \\ 0,10 & 0,20 & -0,30 \end{bmatrix}$$

Ejemplo simple de LoRA: salidas en los tres casos

Sin LoRA:

$$y = Wx = \begin{bmatrix} -0,4 \\ 0,8 \\ -0,9 \end{bmatrix}$$

Con LoRA sin escalado:

$$W_{\text{LoRA}} = W + \Delta W$$

$$y_{\text{LoRA}} = W_{\text{LoRA}}x = \begin{bmatrix} 0,0 \\ 0,48 \\ -0,1 \end{bmatrix}$$

Con LoRA escalado ($\alpha = 32$):

$$\Delta W_{\text{esc}} = \frac{32}{1} \cdot \Delta W$$

$$W_{\text{esc}} = W + \Delta W = \begin{bmatrix} 1,8 & 3,1 & -4,4 \\ -1,28 & -2,26 & 3,64 \\ 2,7 & 6,5 & -9,0 \end{bmatrix}$$

$$y_{\text{esc}} = W_{\text{esc}}x = \begin{bmatrix} 12,4 \\ -9,44 \\ 24,7 \end{bmatrix}$$

Comparativa de parámetros entrenables

Caso general:

Fine-tuning completo: $d \cdot k$ vs LoRA: $r(d + k)$

Ejemplo numérico: $d = k = 2048, r = 8$

Fine-tuning completo: $2048 \times 2048 = 4\,194\,304$ parámetros

LoRA: $8 \cdot (2048 + 2048) = 32\,768$ parámetros

Ahorro: más del 99 % de parámetros, con capacidad de adaptación efectiva

Durante la inferencia, las matrices BA pueden fusionarse con W_0 para no penalizar el rendimiento.

Casos de uso y eficacia empírica de LoRA

- **Alpaca:** adapta LLaMA para tareas de instrucciones usando LoRA. Alto rendimiento con bajo coste computacional.
- **QLoRA:** combina LoRA con cuantización 4-bit. Permite entrenar modelos de hasta 65B parámetros en una GPU.
- **DreamBooth LoRA:** adapta modelos de difusión para generación de imágenes personalizadas.

Conclusión: LoRA puede igualar o superar al *fine-tuning* completo con menor coste, si se ajusta correctamente y se usan buenos datos.

Caso práctico: diseño y datos

Objetivo: Comparar el rendimiento de **modelos clásicos**, un **LLM** y su versión afinada con **LoRA** sobre un conjunto de datos reales del sector bancario.

Modelos comparados:

- Modelos clásicos: regresión logística, árboles de decisión, *random forest*, *XGBoost* y redes neuronales.
- LLM: Flan-T5 base, aplicado a datos tabulares mediante transformación textual.
- LLM + LoRA: versión adaptada del LLM con un bajo coste computacional.

Caso práctico: diseño y datos

Datos utilizados:

- **Origen:** campaña de marketing bancario con el objetivo de ofrecer depósitos a plazo fijo (Portugal, Mayo 2008 – Noviembre 2010).
- **Tamaño total:** 41.188 registros y 21 variables (características del cliente y respuesta a la campaña).
- **Preprocesamiento:**
 - Construcción de la variable temporal Date mediante análisis del **Euribor**.
 - Selección de un subconjunto de 16.349 registros donde el Euribor estaba más bajo y con una proporción más equilibrada entre las clases yes y no de la variable objetivo.
 - Mantenimiento del texto original en inglés para una mayor compatibilidad con el LLM.

Dataset de Kaggle: <https://www.kaggle.com/datasets/henriqueyamahata/bank-marketing>

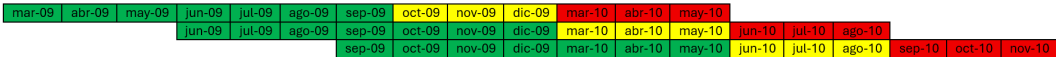
Repositorio GitHub: <https://github.com/Daniseraz/TFG-LLM-LoRA>

División del dataset y validación

Estrategia de particionado temporal:

- División en conjuntos de **train** (7 meses), **validation** (3 meses) y **test** (3 meses), avanzando 3 meses en cada *split*.
- Se emplea **validación cruzada temporal** con ventana deslizante, respetando el orden cronológico para evitar fugas de información.

Esquema visual del particionado:



División del dataset y validación

Distribución de los datos:

	Train	Validation	Test
Split 1	Mar. 09 - Sep. 09 9467 (68.4 %)	Oct. 09 - Dic. 09 1812 (13.1 %)	Mar. 10 - May. 10 2553 (18.5 %)
Split 2	Jun. 09 - Dic. 09 3763 (45.9 %)	Mar. 10 - May. 10 2553 (31.2 %)	Jun. 10 - Ago. 10 1877 (22.9 %)
Split 3	Sep. 09 - May. 10 4572 (64.5 %)	Jun. 10 - Ago. 10 1877 (26.5 %)	Sep. 10 - Nov. 10 640 (9.0 %)

Métricas de evaluación

Matriz de confusión:

	Predice yes	Predice no
Real yes	TP	FN
Real no	FP	TN

En la comparación final se priorizan *precision* y *recall* por su claridad e importancia para controlar falsos positivos y negativos.

Métricas clave:

- **Precision:** proporción de predicciones positivas correctas.

$$\frac{TP}{TP + FP}$$

- **Recall:** proporción de positivos bien detectados.

$$\frac{TP}{TP + FN}$$

- **F1-Score:** media armónica de precisión y *recall*.

$$2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Modelos clásicos de *Machine Learning*

Se han utilizado cinco modelos clásicos: **regresión logística**, **árboles de decisión**, **random forest**, **XGBoost** y **red neuronal**.

Todos los modelos se entrenaron y evaluaron siguiendo el proceso por *split* siguiente:

- 1 Entrenamiento del modelo base con los datos de entrenamiento.
- 2 Evaluación en validación para establecer una referencia inicial.
- 3 Ajuste de hiperparámetros con búsqueda bayesiana, maximizando el *F1-Score* sobre el conjunto de validación.
- 4 Reentrenamiento con los hiperparámetros óptimos sobre el conjunto de entrenamiento.
- 5 Evaluación en entrenamiento y validación para detectar sobreajuste o subajuste.
- 6 Reentrenamiento final usando entrenamiento + validación.
- 7 Evaluación el modelo definitivo en el conjunto de test con todas las métricas.

Este procedimiento se repitió en los tres *splits* definidos, calculando métricas medias por modelo para su comparación posterior con el LLM y LoRA.

Adaptación textual del *dataset* tabular

Para aplicar el modelo Flan-T5 a datos tabulares, cada instancia se transformó en una estructura tipo `instruction-output`, que podemos ver con el siguiente ejemplo:

```
{“instruction”: “age: 18, job: student, marital status: single, education: high.school, ... Will they take out a deposit?”,  
 “output”: “no”}
```

Durante la inferencia, el modelo (base o con LoRA) recibe solo el campo `instruction`. Su predicción se compara con `output` para calcular las métricas.

Diferencia clave:

- **Flan-T5 base:** nunca ve las etiquetas; solo genera respuestas en modo inferencia.
- **Flan-T5 + LoRA:** se entrena usando ejemplos etiquetados para aprender la predicción.

Modelo LLM sin ajuste (Flan-T5 base)

Se evaluó Flan-T5 base sin *fine-tuning*, únicamente con su conocimiento preentrenado. La evaluación se hizo sobre CPU, con tiempos de inferencia entre 1 y 2 horas por *split*.

Procedimiento de inferencia:

- 1 **Entrada textual:** recibe una instrucción con las variables del cliente.
- 2 **Tokenización:** convierte el texto en *input IDs* mediante el tokenizador del modelo.
- 3 **Embeddings:** transforma los tokens en vectores densos con información semántica y posicional.
- 4 **Encoder-Decoder:** procesa la secuencia mediante la arquitectura del modelo T5.
- 5 **Decodificación:** genera una secuencia textual; se toma solo el primer token (yes o no) como predicción final.

Este enfoque sirve como **línea base** para comparar con la versión adaptada mediante LoRA.

Modelo LLM ajustado con LoRA

Para mejorar el rendimiento del modelo base, se aplicó **LoRA**, insertando matrices de bajo rango en capas específicas, manteniendo el resto del modelo congelado. El entrenamiento se realizó sobre CPU y tardó en ejecutarse entre 4 y 5 horas por *split*.

Configuración: `r=16, lora_alpha=32, lora_dropout=0.0, bias="none"`
`target_modules={'q', 'k', 'v', 'o', 'wi', 'wo'}`

Protocolo por split:

- 1 Entrenamiento sobre *train*
- 2 Evaluación en *validation*
- 3 Reentrenamiento con *train + validation*
- 4 Evaluación final sobre *test*

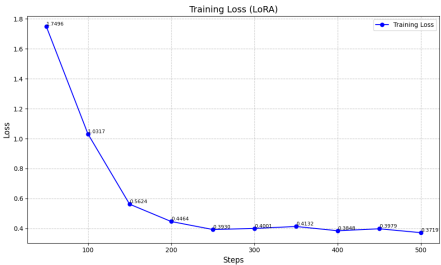
Selección balanceada:

- *train*: 500 ejemplos
(250 yes + 250 no)
- *train+val*: 1000 ejemplos
(500 yes + 500 no)

Modelo LLM ajustado con LoRA

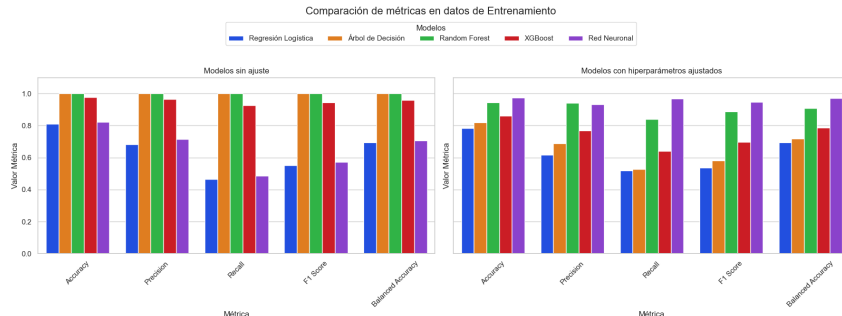
Evolución de la función de pérdida:

Step	Training Loss
50	1.749600
100	1.031700
150	0.562400
200	0.446400
250	0.393000
300	0.400100
350	0.413200
400	0.384800
450	0.397900
500	0.371900



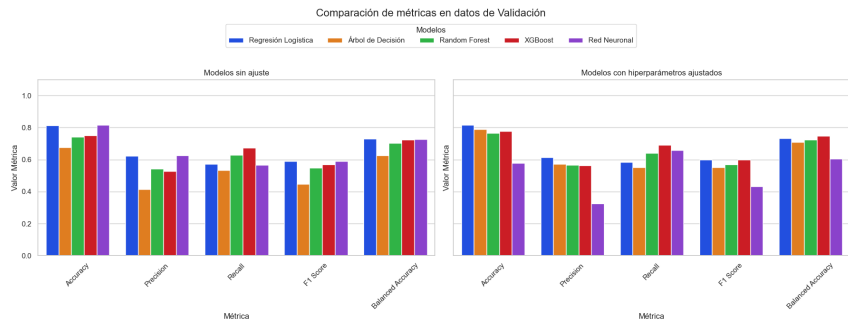
- La función de pérdida disminuye progresivamente durante el entrenamiento, estabilizándose en valores bajos tras las primeras fases, lo que confirma un ajuste efectivo del modelo.
- La salida se interpretó como en el modelo base, permitiendo una comparación directa con métricas comunes.

Comparación en entrenamiento (modelos clásicos)



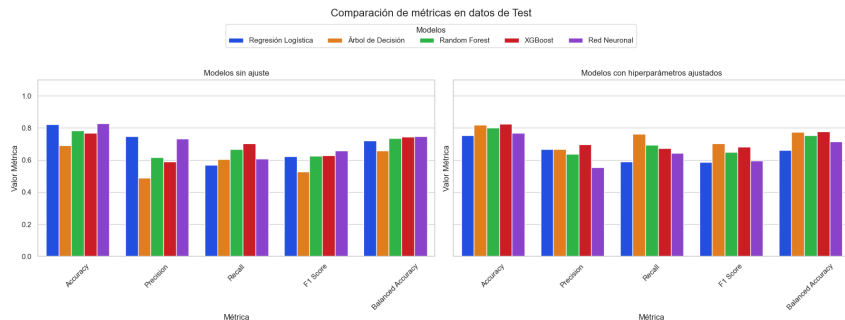
- El ajuste bayesiano permitió una generalización más realista en modelos inicialmente sobreajustados (Árbol de Decisión y *Random Forest*), y potenció el rendimiento de la Red Neuronal.

Comparación en validación (modelos clásicos)



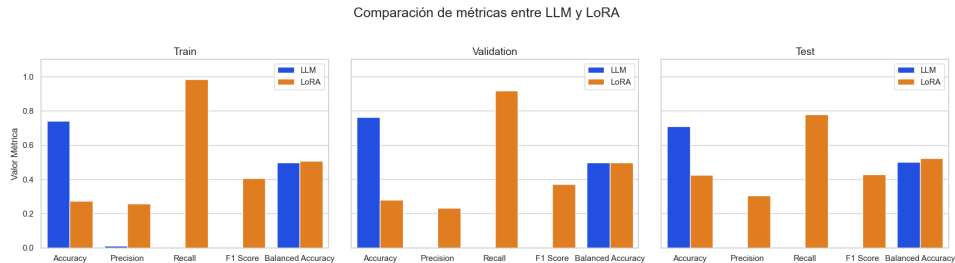
- El ajuste de hiperparámetros mejora claramente el rendimiento de modelos como *Árbol de Decisión* y *XGBoost*. En cambio, la *Red Neuronal* empeora, mientras que *Regresión Logística* y *Random Forest* se mantienen estables. La validación confirma qué mejoras son reales sobre datos no vistos.

Comparación en test (modelos clásicos)



- En test, el ajuste mejora claramente a modelos como *Árbol de Decisión*, *XGBoost* y *Random Forest*. En cambio, la *Red Neuronal* y la *Regresión Logística* empeoran, lo que sugiere que su configuración previa era más adecuada.
- Estos resultados confirman que el ajuste no siempre mejora y debe validarse cuidadosamente.

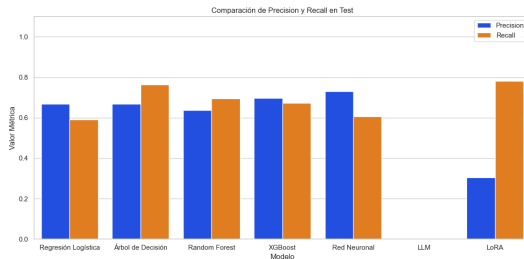
Comparación entre LLM y LLM + LoRA



- El modelo base obtiene buena *Accuracy*, pero su *Precision*, *Recall* y *F1-Score* son muy bajas o nulas, indicando fuerte sesgo hacia la clase mayoritaria. Al aplicar **LoRA**, estas métricas mejoran significativamente, sobre todo el *Recall*, con una ligera pérdida en *Accuracy*, pero mejora en *Balanced Accuracy*.
- LoRA permite adaptar eficazmente un LLM generalista a una tarea concreta, sin necesidad de reentrenar todos sus parámetros.

Comparación final

- Se comparan las métricas *Precision* y *Recall* en test para todos los modelos: clásicos (mejor versión de *F1-Score* en validación), Flan-T5 base y LoRA.



- LoRA destaca por su **alta capacidad de detección** de la clase positiva ($Recall = 0.78$), aunque con una precisión más moderada. El modelo LLM base falla en identificar positivos. Sobresale el Árbol de Decisión seguido por XGBoost y la Red Neuronal siendo estos competitivos en términos de equilibrio general.

Conclusiones

- Este trabajo ha abordado la **optimización de LLMs** mediante técnicas de **ajuste eficiente**, con el objetivo de adaptarlos a tareas específicas sin recurrir a grandes recursos computacionales.
- Se ha combinado una **revisión teórica** rigurosa —incluyendo arquitecturas encoder-decoder, retropropagación y paradigmas de modelado del lenguaje— con un **estudio experimental** sobre datos reales del sector bancario.
- La técnica **LoRA** ha sido el eje central de la parte práctica, demostrando que es posible ajustar modelos de gran tamaño **entrenando solo una fracción de sus parámetros**, manteniendo el resto congelado.
- En comparación con el modelo base, **LoRA mejora significativamente el rendimiento**, incrementando métricas como *recall* y *F1-Score*.
- LoRA permite **adaptar un único LLM preentrenado a múltiples tareas específicas**, sin modificar sus parámetros originales, facilitando el **despliegue eficiente** en entornos con recursos limitados.

Líneas futuras

- **Comparar con otras técnicas PEFT** (*Adapters*, *Prefix Tuning* o *BitFit*) con el mismo modelo base, tarea y datos. Evaluar su rendimiento predictivo, número de parámetros y coste computacional.
- **Extender a modelos más grandes o especializados** (Flan-T5 Large, FinBERT o BloombergGPT). Analizar el impacto de la especialización previa en tareas bancarias.
- **Incluir métricas de eficiencia energética y coste** (ej. codecarbon). Desarrollar métricas coste-beneficio que combinen rendimiento y sostenibilidad.
- **Explorar enfoques híbridos: modelos clásicos + LLMs** (ej. generar variables con LLM y añadirlas a un modelo clásico como *XGBoost* o *Random Forest*). Evaluar si hay una mejora en la interpretabilidad y la capacidad predictiva.

FIN

¡Muchas gracias por su atención!

Técnicas Clásicas de Optimización

■ Cuantización:

- Representa pesos con menos bits (ej. 8-bit).
- Uniforme: $Q(w) = \Delta \cdot \text{round}(w/\Delta)$.
- No uniforme: niveles adaptados a la distribución de pesos.

■ Pruning:

- Elimina pesos con $|w_i| < \tau$, generando un modelo esparso.
- Mejora eficiencia si el hardware soporta tensores dispersos.

■ Destilación de conocimiento:

- Un modelo pequeño (estudiante) aprende de uno grande (profesor).
- Pérdida combinada: $\mathcal{L}_{KD} = \alpha \cdot \mathcal{L}_{\text{hard}} + (1 - \alpha) \cdot \mathcal{L}_{\text{soft}}$.
- Se usa *softmax* suavizado con temperatura T .

Optimización del Proceso de Entrenamiento

■ Warmup y Schedulers:

- Ajuste dinámico de la tasa de aprendizaje (ej. *cosine decay*).
- Mejora estabilidad inicial y convergencia.

■ Batch sizes adaptativos:

- Permiten equilibrar estabilidad, memoria y eficiencia a lo largo del entrenamiento.

■ Gradient checkpointing:

- Reduce uso de memoria recomputando valores intermedios.
- Aumenta ligeramente el tiempo de cómputo.

■ Mixed-precision training:

- Usa FP16 en operaciones comunes y FP32 en puntos críticos.
- Acelera el entrenamiento y reduce consumo de memoria.

Hiperparámetros clave y aplicación de LoRA en Transformers

Hiperparámetros principales:

- `r`: Rango bajo de adaptación. Controla la capacidad del ajuste. Valores típicos: 4–16.
- `alpha`: Escala la contribución de LoRA. Típico: 16–64. Evita valores extremos para estabilidad.
- `lora_dropout`: Regulariza la actualización BA . Útil si hay pocos datos. Sugerido: 0.05–0.1.
- `target_modules`: Define qué matrices se adaptan (ej. "q", "v"). Controla el alcance de LoRA.

Aplicación en Transformers:

- Se aplica en las proyecciones de atención:

$$Q = XW_q, \quad V = XW_v, \dots$$

- Se modifica cada W como:

$$W_q = W_q^0 + \frac{\alpha}{r} B_q A_q$$

- Sólo se entrena BA , manteniendo W^0 fijo.
- Se puede usar en atención y/o feed-forward según la tarea.

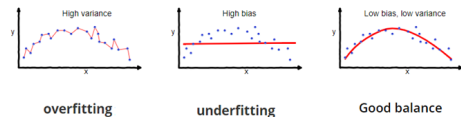
Regularización y sobreajuste

Sobreajuste vs. Subajuste:

- **Sobreajuste:** el modelo memoriza el entrenamiento, falla en generalizar.
- **Subajuste:** el modelo es demasiado simple y no aprende correctamente.

Técnicas de regularización:

- **L2 (Ridge):** penaliza $\|\mathbf{w}\|_2^2$.
- **L1 (Lasso):** penaliza $\|\mathbf{w}\|_1$.
- **Dropout:** desactiva neuronas aleatoriamente.
- **Early stopping:** detiene entrenamiento si la validación empeora.



Representación gráfica de underfitting, good fitting y overfitting.